



Name: ..... Cohort/Batch: ..... Date: ..... Score: .....

DISTRIBUTED TRANSACTIONS PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 conceptual Q&A Distributed Systems

TOTAL: 10 QUESTIONS

**Q1** What is Distributed Transactions and what problem in Distributed Systems does it solve? **Threat Model**

---

---

**Q6** How does Distributed Transactions handle reliability and high availability? **Highly Available**

---

---

**Q2** What are the core components or concepts in Distributed Transactions? **Distributed Architecture**

---

---

**Q7** What observability does Distributed Transactions provide out of the box? **Observability**

---

---

**Q3** How does Distributed Transactions compare to its main alternatives? **Configuration**

---

---

**Q8** What are common performance bottlenecks in Distributed Transactions? **Performance**

---

---

**Q4** How do you get started with Distributed Transactions for a new project? **Getting Started**

---

---

**Q9** What security best practices apply when running Distributed Transactions? **Security**

---

---

**Q5** What configuration options most affect Distributed Transactions's behavior in production? **SSO / IdP**

---

---

**Q10** What mistakes do teams commonly make when adopting Distributed Transactions? **Tuning**

---

---



ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 Distributed Transactions is a tool or platform

Mitigation

Distributed Transactions is a tool or platform that automates or simplifies a specific concern in the Distributed Systems domain, reducing manual effort and operational complexity for engineering teams.

A6 Distributed Transactions provides HA through leader election

Automation

Distributed Transactions provides HA through leader election, replication, or stateless horizontal scaling. Deploy at least two instances across availability zones and configure health checks for automatic failover.

A2 Distributed Transactions has key building blocks that

Primitives

Distributed Transactions has key building blocks that work together: a configuration layer defines intent, a runtime layer enforces it, and an API or CLI layer allows operators to manage and observe the system.

A7 Distributed Transactions exposes metrics (Prometheus)

Automation

Distributed Transactions exposes metrics (Prometheus endpoint or built-in dashboard), structured logs, and optionally distributed traces. Define SLOs on the key metrics and alert before users are affected.

A3 Distributed Transactions trades off simplicity against feature

Feature

Distributed Transactions trades off simplicity against feature richness differently than its alternatives. Choose Distributed Transactions when its specific strengths performance, ecosystem, or ease of use align with your team's primary constraints.

A8 Performance bottlenecks typically stem from misconfigured

Performance

Performance bottlenecks typically stem from misconfigured connection pools, large payload sizes, insufficient worker threads, or missing caches. Profile with built-in diagnostics before optimizing.

A4 Install Distributed Transactions via its package manager

Gotchas

Install Distributed Transactions via its package manager or binary release. Follow the quickstart to initialize a project, configure the minimal required settings, and run the default command to verify the setup works end-to-end.

A9 Run Distributed Transactions with the principle of

Assessment

Run Distributed Transactions with the principle of least privilege, enable TLS for all communication, use a secrets manager for credentials, and keep the software patched to the latest stable release.

A5 Production-critical settings typically include concurrency

Concurrency

Production-critical settings typically include concurrency limits, timeout values, retry policies, resource limits, and logging levels. Review the official production hardening guide before deploying.

A10 Common pitfalls include skipping the documentation and

Documentation

Common pitfalls include skipping the documentation and learning the API by trial and error, using default configurations in production, lacking a runbook for operational issues, and not testing failover scenarios.